

Zero Trust Secure Communication Framework — FULL Evaluation Master Guide

Project Title

Zero Trust Secure Communication Framework for Malware Sample Submission Using Mutual Authentication, End-to-End Encryption and Post-Quantum Readiness

1. SIMPLE PROJECT INTRODUCTION

What is our project?

Our project is a secure communication framework designed to safely submit malware binaries from a client to a malware analysis server.

Since malware samples are highly sensitive and dangerous files, sending them using normal HTTP or unsecured communication introduces major risks such as:

- Malware interception
- Malware tampering
- Unauthorized submissions
- Client/server impersonation
- Replay attacks
- Future quantum-computing threats

To solve these problems, our system uses:

- Mutual TLS (mTLS)
 - AES-256-GCM encryption
 - HMAC-SHA256 integrity verification
 - RSA-PSS digital signatures
 - Hybrid ECDH + Kyber-768 post-quantum key exchange
-

2. MAIN OBJECTIVE

The main objective of our system is:

To create a Zero Trust secure communication framework that securely transmits malware samples between a client and malware analysis server while ensuring authentication, confidentiality, integrity, non-repudiation, and post-quantum readiness.

3. LINK TO FYP

Our Final Year Project focuses on malware static analysis using graph-based techniques and machine learning.

However, before malware analysis happens, malware binaries first need to be securely transmitted to the analysis server.

This Information Security project secures that malware submission pipeline.

Integration Flow

Analyst/User

↓

Secure Submission Framework (THIS PROJECT)

↓

Verified & Decrypted Malware Sample

↓

Malware Analysis Pipeline (FYP)

↓

CFG / GNN / Classification

Important sentence:

Our Information Security project acts as a secure gateway in front of the malware analysis infrastructure.

4. SYSTEM ARCHITECTURE

Full Architecture Flow

CLIENT

↓

mTLS Authentication

↓

ECDH + Kyber Hybrid Key Exchange

↓

HKDF Session Key Derivation

↓

AES-256-GCM Encryption

↓

HMAC-SHA256 Integrity Protection

↓

RSA-PSS Digital Signature

↓

POST /submit

↓

SERVER Verification

↓

Decryption

↓

Audit Logging

↓

Secure Response

5. COMPONENT-BY-COMPONENT EXPLANATION

5.1 Certificates and CA

What is it?

Certificates prove identity.

The CA (Certificate Authority) signs certificates for:

- Server
- Client

Why we use it

To ensure:

- only trusted clients can connect
- clients communicate with the real server

Security Property

Authentication

Files Used

certs/ca.crt

certs/server.crt

certs/client.crt

Important Viva Sentence

We use a private CA to issue certificates to both client and server, enabling mutual authentication.

5.2 Mutual TLS (mTLS)

What is mTLS?

Normal HTTPS only authenticates the server.

mTLS authenticates:

- Client
- Server

Both verify each other.

Why we use it

Because malware submission must only be allowed from trusted clients.

Security Property

Mutual Authentication

Code Location

server/app.py

Important line:

```
ctx.verify_mode = ssl.CERT_REQUIRED
```

Viva Answer

mTLS ensures that both client and server verify each other's identity before communication begins.

5.3 ECDH — Elliptic Curve Diffie-Hellman

What is ECDH?

ECDH is a secure key agreement algorithm.

It allows both sides to derive the same shared secret without transmitting the secret itself.

Why we use it

To securely derive a session key.

Security Property

- Secure key exchange
- Forward secrecy

What is Forward Secrecy?

Even if long-term server keys are stolen later, previous session keys remain safe.

Code Location

common/crypto_utils.py

Functions:

```
generate_ecdh_keypair()
```

ecdh_derive()

Viva Answer

We use ephemeral ECDH to derive shared secrets securely while providing forward secrecy.

5.4 Kyber-768 / ML-KEM

What is Kyber?

Kyber is a post-quantum key encapsulation mechanism.

New standard name:

ML-KEM

Why we use it

Quantum computers may break ECC and RSA using Shor's Algorithm.

Kyber is lattice-based and designed to resist quantum attacks.

Security Property

Post-Quantum Readiness

Code Location

common/crypto_utils.py

Functions:

kyber_keygen()

kyber_encapsulate()

kyber_decapsulate()

Important Viva Sentence

Kyber was standardized by NIST under FIPS 203 as a post-quantum key encapsulation mechanism.

5.5 Hybrid Key Exchange

What is Hybrid Key Exchange?

We combine:

ECDH shared secret

+

Kyber shared secret

Then derive one final AES key.

Why we use it

If ECDH breaks:

- Kyber still protects

If Kyber breaks:

- ECDH still protects

Security Property

Defence-in-depth and migration security.

Important Formula

$\text{HKDF}(\text{ECDH} || \text{Kyber}) \rightarrow \text{AES-256 Key}$

Viva Answer

The session remains secure as long as at least one of the two key exchange mechanisms remains secure.

5.6 HKDF

What is HKDF?

HKDF means:

HMAC-based Key Derivation Function

It converts raw shared secrets into a secure AES key.

Why we use it

Raw secrets should not directly become AES keys.

HKDF:

- removes bias
- standardizes randomness
- derives correct key size

Output

32-byte AES-256 key

Viva Answer

HKDF securely derives the final AES-256 session key from the combined ECDH and Kyber shared secrets.

5.7 AES-256-GCM

What is AES-GCM?

AES is symmetric encryption.

GCM is an authenticated encryption mode.

Why we use it

To encrypt the malware binary.

Security Properties

- Confidentiality
- Integrity

Important Values

AES Key:

256 bits

Nonce:

96 bits

Code Location

common/crypto_utils.py

Functions:

encrypt_binary()

decrypt_binary()

Viva Answer

AES-256-GCM provides authenticated encryption, meaning it protects both confidentiality and integrity simultaneously.

5.8 HMAC-SHA256

What is HMAC?

HMAC is a message authentication code.

It detects tampering.

Why we use it

Defence-in-depth.

Even though AES-GCM already authenticates, HMAC provides independent application-layer integrity.

Security Property

Integrity

Code Location

common/crypto_utils.py

Important Function

hmac.compare_digest()

Used to prevent timing attacks.

Viva Answer

HMAC-SHA256 provides additional application-layer integrity verification over nonce and ciphertext.

5.9 RSA-PSS Digital Signatures

What is Digital Signature?

Client signs metadata using private RSA key.

Server verifies using public RSA key.

Why we use it

To provide:

- Non-repudiation
- Identity verification

Why sign metadata instead of full binary?

Large binaries are expensive to sign.

Instead we sign:

- SHA256 hash
- timestamp
- client identity

Security Property

Non-repudiation

Viva Answer

RSA-PSS signatures ensure the client cannot deny submitting a malware sample.

5.10 SHA-256

What is SHA-256?

A cryptographic hash function.

Creates a unique fingerprint of the malware sample.

Why we use it

To uniquely identify submitted binaries.

Viva Answer

SHA-256 generates a unique fingerprint used for audit logging and malware identification.

5.11 Audit Logging

What is Audit Logging?

Server records:

- client identity
- timestamp
- SHA256 hash
- submission status

Why we use it

For:

- accountability
- forensic analysis
- traceability

Viva Answer

Audit logs provide accountability and traceability for malware submissions.

6. THREAT MODEL

Threat 1 — Eavesdropping

Attack

Attacker intercepts malware sample.

Mitigation

- TLS 1.3
- AES-256-GCM

Security Property

Confidentiality

Threat 2 — Tampering

Attack

Attacker modifies ciphertext.

Mitigation

- HMAC-SHA256
- AES-GCM authentication tag

Security Property

Integrity

Threat 3 — Impersonation

Attack

Fake client pretends to be authorized user.

Mitigation

- Mutual TLS
- CA-signed certificates

Security Property

Authentication

Threat 4 — Forgery

Attack

Attacker submits malware pretending to be another user.

Mitigation

- RSA-PSS signatures

Security Property

Non-repudiation

Threat 5 — Replay Attack

Attack

Attacker replays old submission.

Mitigation

- ephemeral ECDH
- unique nonces

Security Property

Freshness

Threat 6 — Quantum Threat

Attack

Quantum computers break ECC/RSA.

Mitigation

- Hybrid ECDH + Kyber-768

Security Property

Post-Quantum Readiness

7. DEMONSTRATION FLOW

Step 1 — Start Server

```
python server/app.py
```

Expected Output:

Running on https://127.0.0.1:5000

Explanation:

The Flask server starts with TLS enabled and requires mandatory client certificate verification.

Step 2 — Run Happy Path Demo

python demo.py

Expected Output

[1] PQ KEM backend initialized

[2] Starting mTLS + hybrid ECDH/Kyber key exchange

[3] AES-256 session key derived using HKDF(ECDH || Kyber)

[4] Submitting encrypted payload + HMAC + RSA-PSS signature

[5] Server response HTTP 200

Explanation

Step 1

mTLS authentication begins.

Step 2

ECDH and Kyber exchange shared secrets.

Step 3

HKDF derives AES session key.

Step 4

Malware binary encrypted using AES-GCM.

Step 5

Server verifies:

- HMAC
- RSA signature
- GCM tag

Then decrypts and logs malware.

Step 3 — Attack Demonstrations

python attack_demos.py

Attack 1 — No Client Certificate

Expected Result

TLS handshake failure.

Explanation

The server rejects unauthorized clients before communication starts.

Attack 2 — Tampered Payload

Expected Result

HMAC verification failure.

Explanation

Modified ciphertext causes integrity validation failure.

Attack 3 — Forged Signature

Expected Result

RSA-PSS signature verification failure.

Explanation

Invalid signatures prevent impersonation.

Final Happy Path

Valid authenticated submission succeeds.

8. POST-QUANTUM DISCUSSION

Why Post-Quantum Security?

Quantum computers may break:

- RSA
- ECC

- ECDH

using:

Shor's Algorithm

Our Solution

We implemented:

Hybrid ECDH + Kyber-768

Why Hybrid?

If ECDH breaks:

- Kyber protects

If Kyber breaks:

- ECDH protects

Important Sentence

The AES session key remains secure as long as either the classical ECDH or the post-quantum Kyber mechanism remains secure.

9. LIMITATIONS

Limitation 1

Prototype-scale implementation.

Limitation 2

No distributed PKI.

Limitation 3

No certificate revocation system.

Limitation 4

Single-node Flask server.

Limitation 5

Educational Kyber implementation.

10. FUTURE IMPROVEMENTS

Possible future work:

- Distributed secure infrastructure
- Hardware Security Modules
- SIEM integration
- Production-grade PQC libraries
- Kubernetes deployment
- Secure sandbox integration
- Analyst dashboard
- Automated malware detonation

11. FINAL CONCLUSION

In conclusion, we developed a Zero Trust secure malware submission framework integrating mutual TLS authentication, AES-256-GCM encrypted communication, HMAC integrity verification, RSA-PSS digital signatures, and hybrid ECDH plus Kyber-768 post-quantum key exchange to provide secure and future-resistant communication for malware analysis environments.

12. IMPORTANT VIVA QUESTIONS

Why AES-GCM instead of AES-CBC?

AES-GCM provides authenticated encryption and avoids padding oracle vulnerabilities.

Why HMAC if GCM already authenticates?

Defence-in-depth and application-layer integrity.

Why ECDH?

Secure key exchange with forward secrecy.

Why Kyber?

Post-quantum security against future quantum computers.

Why Hybrid?

Session remains secure if either ECDH or Kyber remains secure.

Why RSA-PSS?

More secure than PKCS#1 v1.5.

What is Zero Trust?

No implicit trust.

Every request requires authentication and verification.

What is Forward Secrecy?

Old sessions remain safe even if long-term keys are compromised.

What is Non-repudiation?

The sender cannot deny submitting the malware sample.

Why use SHA-256?

To uniquely identify malware binaries.

13. FINAL QUICK FLOW TO MEMORIZE

Client

↓

mTLS Authentication

↓

ECDH + Kyber Key Exchange

↓

HKDF Session Key Derivation

↓

AES-256-GCM Encryption

↓

HMAC Integrity Verification

↓

RSA-PSS Signature

↓

Secure Malware Submission

↓

Server Verification & Decryption

↓

Audit Logging

↓

Secure Response

14. FULL EVALUATION SPEECH FLOW

Opening

Good morning sir/madam.

Our project is titled:

"Zero Trust Secure Communication Framework for Malware Sample Submission Using Mutual Authentication, End-to-End Encryption and Post-Quantum Readiness."

The main objective of our project is to securely transmit malware binaries from a client machine to a malware analysis server while ensuring authentication, confidentiality, integrity, non-repudiation, and post-quantum readiness.

In malware analysis systems, malware binaries are highly sensitive and dangerous files. If such binaries are transmitted using normal communication channels, attackers may intercept, tamper with, replay, or forge submissions.

Additionally, future quantum computers may break traditional public key cryptography such as RSA and ECC.

To solve these issues, we designed a Zero Trust secure communication framework using:

- Mutual TLS authentication
- Hybrid ECDH + Kyber-768 key exchange
- HKDF-based session key derivation
- AES-256-GCM encryption
- HMAC-SHA256 integrity verification
- RSA-PSS digital signatures

This project is directly linked with our Final Year Project because our FYP performs malware static analysis, and this Information Security project secures the malware submission pipeline before malware reaches the analysis server.

Architecture Explanation Speech

Now I will explain the system architecture.

Our system contains two main entities:

- Client
- Malware analysis server

The communication begins with mutual TLS authentication.

Both the client and server present X.509 certificates signed by a trusted Certificate Authority.

This ensures:

- the client verifies the real server
- the server verifies the authorized client

This follows the Zero Trust principle where no entity is trusted automatically.

After authentication, the client and server perform hybrid key exchange.

First, they generate ephemeral ECDH key pairs and derive a classical shared secret.

Then the system performs Kyber-768 post-quantum key encapsulation to derive a post-quantum shared secret.

Both shared secrets are combined using HKDF.

The final AES-256 session key is derived using:

HKDF(ECDH || Kyber)

This hybrid approach ensures the session remains secure as long as either ECDH or Kyber remains secure.

Next, the malware binary is encrypted using AES-256-GCM.

AES-GCM provides authenticated encryption, meaning it protects both confidentiality and integrity.

Additionally, we generate an HMAC-SHA256 value over the nonce and ciphertext for defence-in-depth application-layer integrity verification.

Then the client signs the malware metadata using RSA-PSS digital signatures.

This provides non-repudiation because the sender cannot deny submitting the malware sample.

Finally, the encrypted payload, HMAC, signature, and metadata are submitted to the Flask server through the secure TLS channel.

The server verifies:

- HMAC integrity
- RSA signature
- AES-GCM authentication tag

before decrypting the malware sample.

After successful verification, the server generates the SHA-256 hash of the malware sample and stores an audit log entry.

Threat Model Speech

Now I will explain the threat model and mitigations.

We considered multiple threats including:

- Eavesdropping
- Tampering
- Impersonation
- Forgery
- Replay attacks
- Quantum threats

To prevent eavesdropping, we use TLS 1.3 together with AES-256-GCM encrypted payload transmission.

To prevent tampering, we use HMAC-SHA256 and AES-GCM authentication tags.

To prevent impersonation, we use mutual TLS authentication with CA-signed certificates.

To prevent forgery, we use RSA-PSS digital signatures.

Replay attacks are mitigated using ephemeral ECDH sessions and unique nonces.

Finally, future quantum threats are mitigated using hybrid ECDH plus Kyber-768 key exchange.

Demonstration Speech

Now I will demonstrate the system.

First, I start the secure Flask server.

The server uses TLS with mandatory client certificate verification.

Then I run the normal happy-path demonstration.

The client performs:

- mTLS authentication
- ECDH + Kyber key exchange
- HKDF session key derivation
- AES-256-GCM encryption
- HMAC generation
- RSA-PSS signing

before securely submitting the malware sample.

The server then verifies all security controls and accepts the malware sample.

Next, I run the attack demonstrations.

In the first attack, the client attempts connection without a valid certificate.

The TLS handshake fails because the server enforces mandatory client authentication.

In the second attack, the encrypted payload is modified.

HMAC verification fails, proving integrity protection works correctly.

In the third attack, metadata is signed using an invalid private key.

RSA-PSS verification fails, preventing impersonation.

Finally, the valid happy-path submission succeeds again, proving that the system blocks malicious requests while accepting legitimate clients.

Post-Quantum Discussion Speech

One of the most important features of our system is post-quantum readiness.

Traditional public key cryptography such as RSA and ECC may become vulnerable once large-scale quantum computers become practical.

Quantum computers can break these systems using Shor's Algorithm.

To address this problem, we implemented Kyber-768, also known as ML-KEM, together with classical ECDH in a hybrid architecture.

Kyber was standardized by NIST under FIPS 203 as a post-quantum key encapsulation mechanism.

Our hybrid design ensures that the session remains secure as long as either ECDH or Kyber remains secure.

Limitations Speech

Currently, the system is implemented as a prototype-scale educational framework.

The architecture does not yet include:

- distributed deployment
- certificate revocation systems
- production-grade PQC libraries
- high-availability infrastructure

However, the current implementation successfully demonstrates Zero Trust secure communication with post-quantum readiness.

Future Improvements Speech

Future work may include:

- distributed secure malware analysis infrastructure
 - SIEM integration
 - secure sandbox detonation
 - Kubernetes deployment
 - Hardware Security Modules
 - production-grade post-quantum cryptography libraries
 - analyst dashboard interfaces
-

Final Conclusion Speech

In conclusion, we developed a Zero Trust secure malware submission framework integrating mutual TLS authentication, AES-256-GCM encrypted communication, HMAC

integrity verification, RSA-PSS digital signatures, and hybrid ECDH plus Kyber-768 post-quantum key exchange.

Our project demonstrates defence-in-depth secure communication capable of protecting malware submission pipelines against both current and future threats.

Thank you.

14. GOLDEN FINAL SENTENCE

Our project demonstrates a defence-in-depth Zero Trust architecture where authentication, confidentiality, integrity, non-repudiation, and post-quantum readiness are combined to securely protect malware submission pipelines against both current and future threats.